

EMBEDDED MACHINE LEARNING · PHD COURSE

SESSION 04

Hardware platforms and deployment

What the processor actually does with a network, the landscape from microcontrollers to neuromorphic chips, and the path from a trained graph to a measured, monitored binary.

THE ORGANISING QUESTION

Which reuse does this machine's dataflow capture — and does my layer have that kind of reuse?

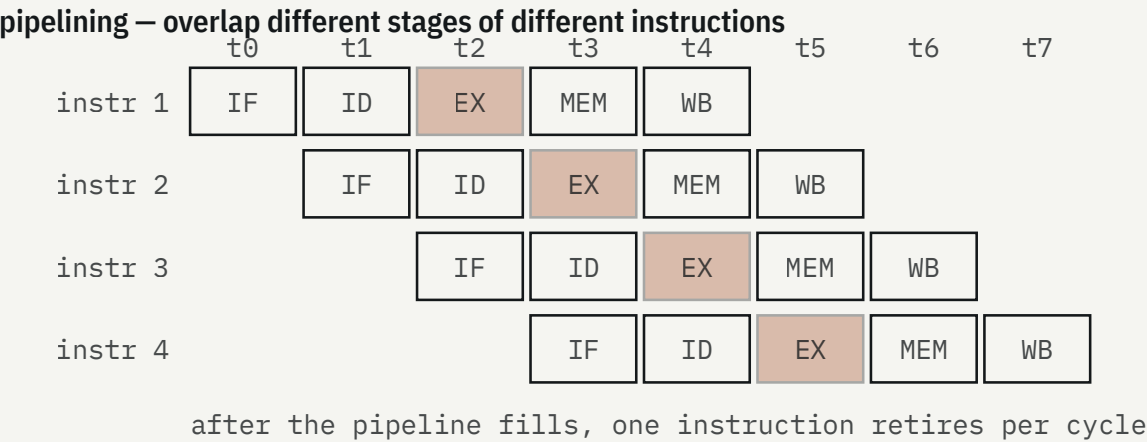
That question predicts performance far better than peak TOPS.
Keep it in your head for the whole session.

THE THESIS

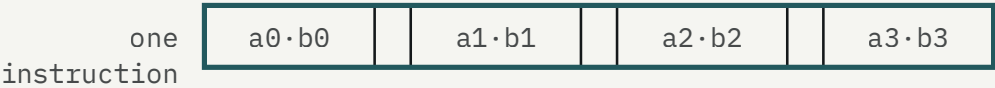
A fast matrix multiply is not a cleverer algorithm. It is the same 2mnk multiply–accumulates, reorganised so each operand is loaded once and used many times.

WHAT AN EMBEDDED CORE CAN STILL OFFER

Pipelining and SIMD



SIMD — one instruction, several data elements



Cortex-M4 SMLAD: 2 × int16 MACs per cycle

Helium: 16 × int8

Out-of-order issue, speculation and deep caches are **absent** on a Cortex-M0+ or M4.

01

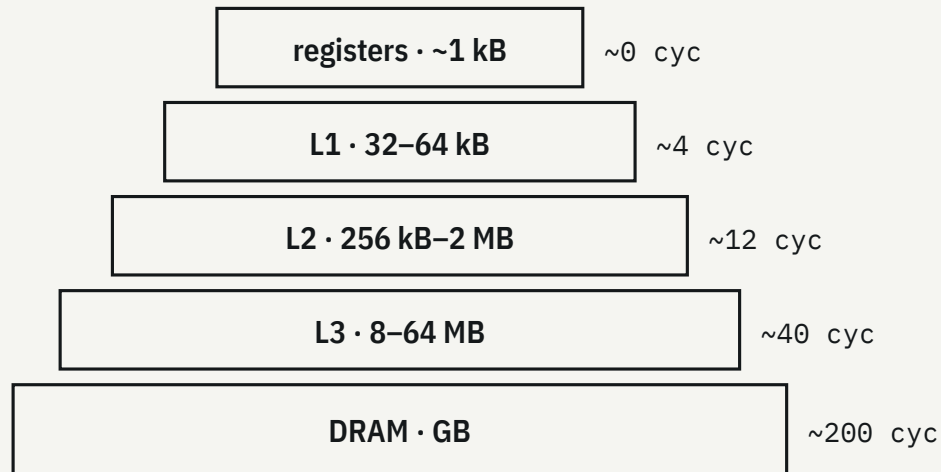
The memory hierarchy

Big and fast is not something you buy. It is something you arrange.

TWO MACHINES, ONE PRINCIPLE

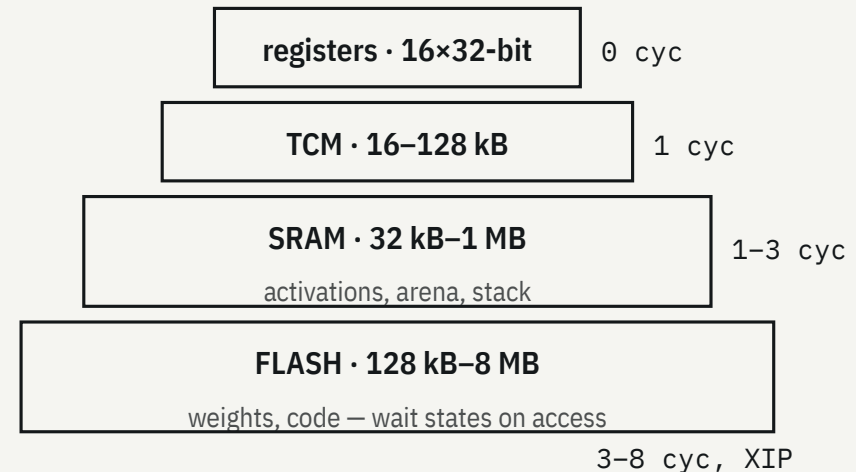
Who decides what is close?

application CPU — hardware-managed hierarchy



you tune by **hinting**: blocking, prefetch, layout

Cortex-M class MCU — software-managed, mostly flat



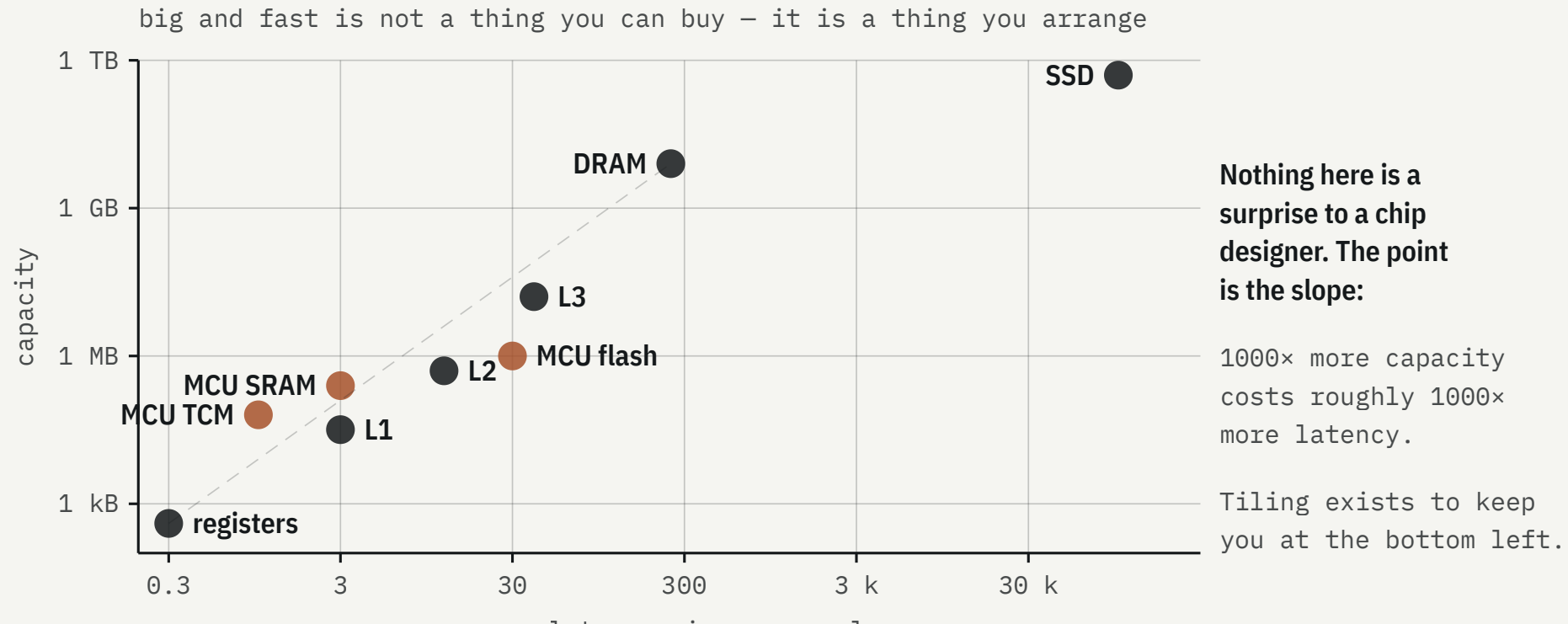
you tune by **placing**: which array lives in which memory

Same principle — keep operands close — but on an MCU it is an explicit linker-script decision, not a cache heuristic.

On a CPU the hardware decides and you **hint**. On an MCU **you** decide, in the linker script.

THE TRADE, PLOTTED

1000× more capacity costs about 1000× more latency



Tiling exists to keep you at the bottom left of this line.

FOUNDATIONS

Locality, in two rules

Memory is delivered in LINES (32 or 64 bytes), never in words.

Spatial locality → touch the rest of a line before it is evicted

Temporal locality → if you will need a value again, use it again soon

A microcontroller without a cache does not escape this; it inherits a harsher version, because no hardware guesses for you. The compensation is predictability: cycle counts on paper actually match the device.

02

The roofline model

A bound, not a prediction. It tells you what is impossible and how much headroom is left.

DERIVATION

Two lower bounds on time

$$I = W / Q \quad \text{arithmetic intensity (ops per byte)}$$

$$T \geq \max(W/\pi , Q/\beta) \quad \text{compute bound, memory bound}$$

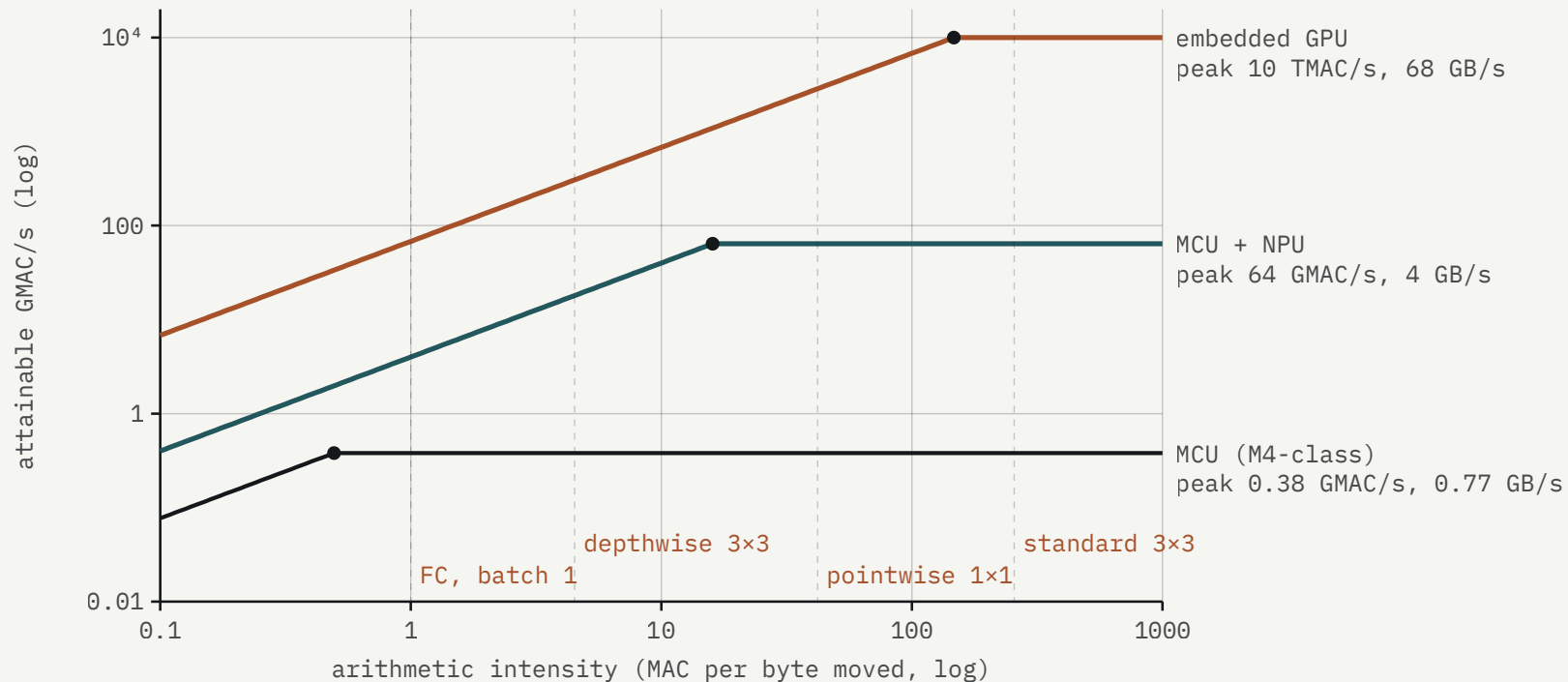
$$P = W/T \leq \min(\pi , \beta \cdot I)$$

$$\text{ridge point } I^* = \pi / \beta$$

π is peak compute, β is peak bandwidth. The ridge point is the arithmetic intensity a kernel must exceed to have any chance of reaching peak compute.

THREE DEVICES, ONE PLOT

The same layer has a different bottleneck on each device



Illustrative int8 rooflines: order-of-magnitude figures, not datasheet guarantees.
Layer intensities computed for int8 tensors of 48×48×64 (FC: weights read once).

THE ANSWER

Arithmetic intensity, computed for a 14×14×256 int8 activation

LAYER	MACS	BYTES MOVED	ARITHMETIC INTENSITY
Standard 3×3, 256→256	1.16×10^8	6.9×10^5	$\approx 168 \text{ MAC/B}$
Depthwise 3×3, 256	4.5×10^5	1.0×10^5	$\approx 4.4 \text{ MAC/B}$
Pointwise 1×1, 256→256	1.28×10^7	1.7×10^5	$\approx 77 \text{ MAC/B}$
Dense, 1024→1024	1.05×10^6	1.05×10^6	$\approx 1 \text{ MAC/B}$

AI = MACs / (bytes in + bytes of weights + bytes out), at batch 1. Depthwise and dense read almost as many bytes as they perform multiplies.

TWO WORKED RIDGE POINTS

The bar is much lower on a microcontroller

LAPTOP CPU CORE	CORTEX-M4F @ 80 MHZ	DENSE LAYER, BATCH 1	DEPTHWISE 3×3
5	0.5	1	4.4
≈100 GFLOP/s over ≈20 GB/s → I* = 5 FLOP/byte	≈160 MMAC/s over ≈320 MB/s → I* = 0.5 MAC/byte	≈1 MAC/byte — hopeless on the laptop, fine on the MCU	MAC/byte — above the M4F ridge, below a GPU's

A genuinely useful asymmetry: **on an MCU, reducing MACs often really does reduce time.**

03

GEMM in five steps

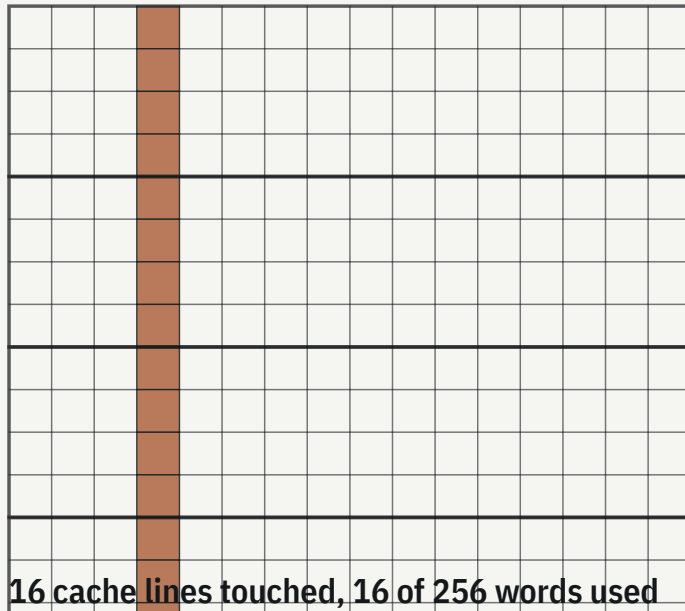
From the naive triple loop to a production kernel.

STEP 2, MADE VISIBLE

Same arithmetic, four times the traffic

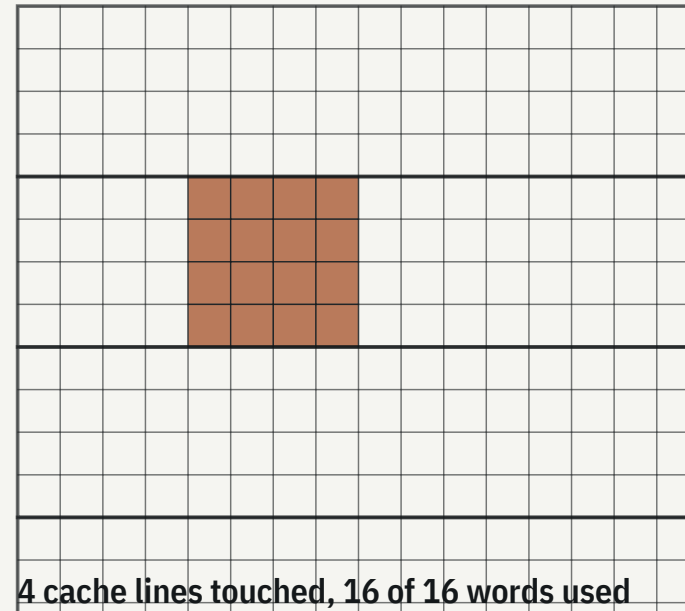
naive inner loop: $B[r, j]$

one column, strided by a full row



blocked inner loop: a 4×4 tile

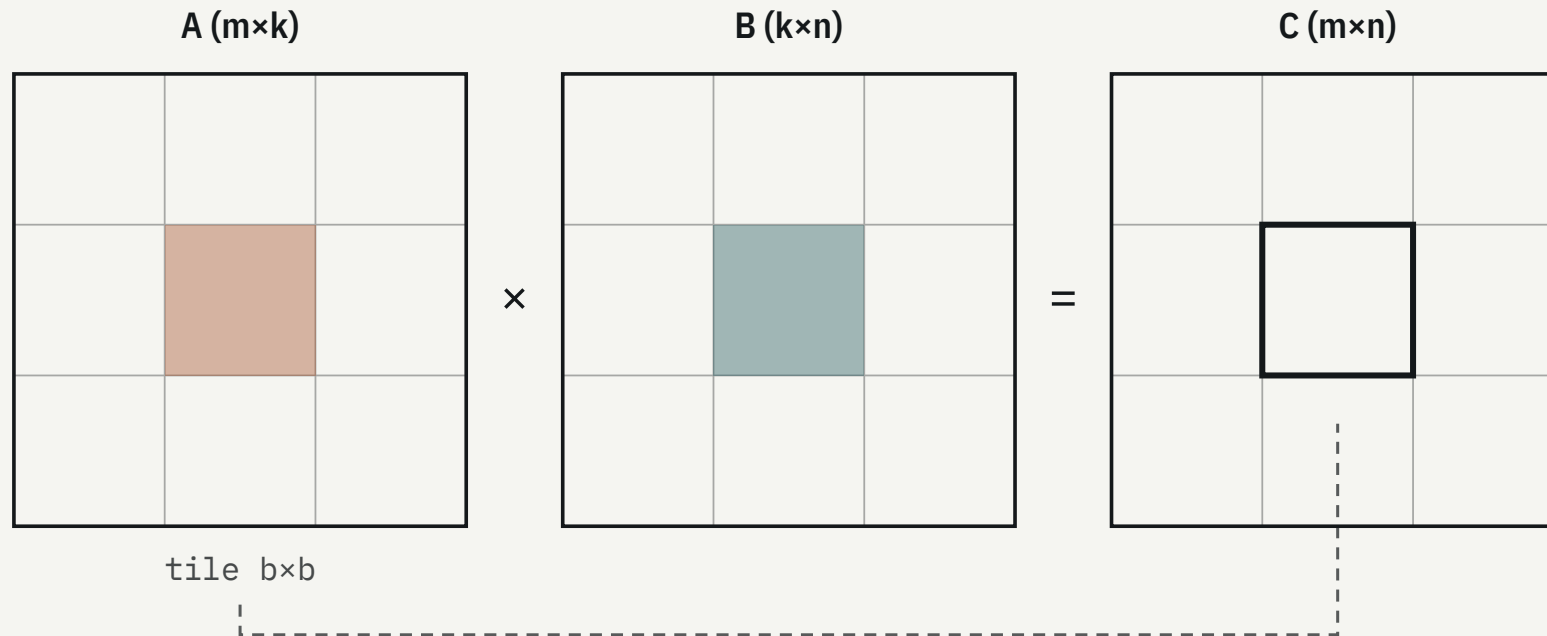
contiguous within each line



Grey rectangles are cache lines. Copper cells are the words the loop actually uses.

WHY BLOCKING WORKS

Load $2b^2$ elements, do b^3 multiply-accumulates



loaded once:
 $2b^2$ elements

work done:
 b^3 MACs

reuse factor = $b/2$

bigger tile $\rightarrow$ more reuse,
until it stops fitting

Arithmetic intensity scales with b – and b is capped by the fast memory that must hold three tiles.

CONVOLUTION AS GEMM

im2col: reuse the fast kernel, pay in memory

input 4×4, kernel 3×3 → 2×2 outputs

0	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15

copy



0	1	4	5
1	2	5	6
2	3	6	7
4	5	8	9
5	6	9	10
6	7	10	11
8	9	12	13
9	10	13	14
10	11	14	15

patches: $K^2 \cdot C_{in} \times (H_{out} \cdot W_{out})$

filters, flattened

$C_{out} \times K^2 \cdot C_{in}$

×

= output

$C_{out} \times (H_{out} \cdot W_{out})$

one GEMM → a tuned GEMM kernel does the work
cost: the patch matrix duplicates each input
value up to $K^2 = 9$ times in memory

MCU libraries (e.g. CMSIS-NN) build only a few
columns at a time to keep that copy small.

CONVOLUTION ON A CPU

Six strategies, two of which change the numerics

STRATEGY	ARITHMETIC	EXTRA MEMORY	NOTE
im2col + GEMM	baseline	$\times k^2$ patch buffer	Reuses a tuned GEMM; the blow-up is often fatal on an MCU
Partial im2col	baseline	two columns	CMSIS-NN’s approach — bounded SRAM
Implicit GEMM	baseline	none	Index arithmetic replaces the copy; standard on GPUs
Direct convolution	baseline	none	Best for depthwise
Winograd F(2×2,3×3)	16 vs 36 mults → 2.25×	transform buffers	Numerically fragile in low precision
FFT convolution	wins for large k	complex buffers	Rarely worth it for 3×3; relevant for long 1-D audio kernels

Layout matters as much as algorithm: **NHWC** keeps channels innermost and contiguous, which is what a vectorised inner product over channels wants.

01

The platform landscape

What each accelerator accelerates — and what it demands of the model.

FOUR FEATURES THAT DECIDE YOUR DESIGN

Planning an MCU inference

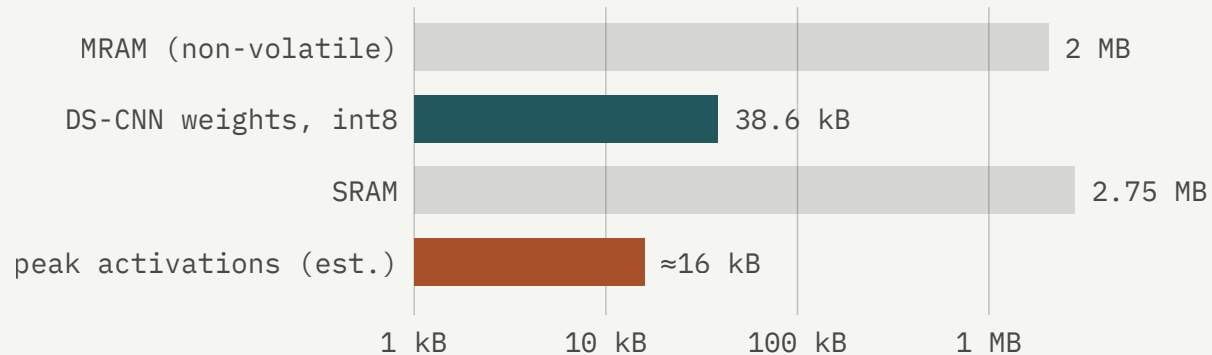
- **A flat, explicit memory map.** No virtual memory, an MPU at most. You choose — in the linker script — what lives in flash, SRAM or TCM.
- **DMA.** Acquire the next audio buffer while computing on the previous one. Double buffering is the difference between meeting a deadline and dropping frames.
- **Power states.** Run, sleep, deep sleep, retention — with wake-up latencies from microseconds to milliseconds. **Wake-up energy frequently dominates the compute.**
- **Interrupt-driven acquisition.** A short, bounded ISR; inference in the main loop. Get this wrong and you get sporadic sample loss that looks exactly like a model problem.

Matrix multiply here is Session 4 with the constants changed: no cache to block for, partial im2col, int8 weights with int32 accumulators, SMLAD inner loops.

CASE STUDY

A keyword spotter on an Apollo4-class MCU, every assumption visible

Memory: the model is tiny compared with this device



Energy per inference: a worked estimate (assumptions stated)

work	≈ 2.7 M MACs per inference (5.4 M ops, Zhang et al. 2017)
throughput	assume ≈ 1 MAC per cycle sustained with SIMD (int8)
time	2.7 M cycles ÷ 192 MHz ≈ 14 ms
current	4 μA/MHz × 192 MHz ≈ 0.77 mA (vendor "as low as" figure)
charge	0.77 mA × 14 ms ≈ 11 μC
energy	× ≈1.8 V supply ≈ 20 μJ per inference, core only

What the estimate leaves out

- MFCC front end
- microphone, PDM interface
- SRAM retention in sleep
- radio, if results are sent
- measured kernel efficiency

At one inference per second:

$20 \mu\text{J} \times 1 \text{ Hz} = 20 \mu\text{W}$
a quarter of the coin
cell budget (Session 1)

THE THIRD CORE IN THE ROOM

Digital signal processors

- **Harvard architecture** — separate instruction and data buses, so a MAC can fetch an operand and an instruction in the same cycle.
- Single-cycle multiply–accumulate with wide **guard-bit accumulators**.
- **Hardware circular addressing** — free ring buffers for FIR filters and streaming windows.
- Zero-overhead loops, and often **VLIW** issue scheduled statically by the compiler.
- For an always-on audio front end — filtering, FFT, mel filterbank — a DSP core is frequently more efficient than either the MCU or the NPU beside it.

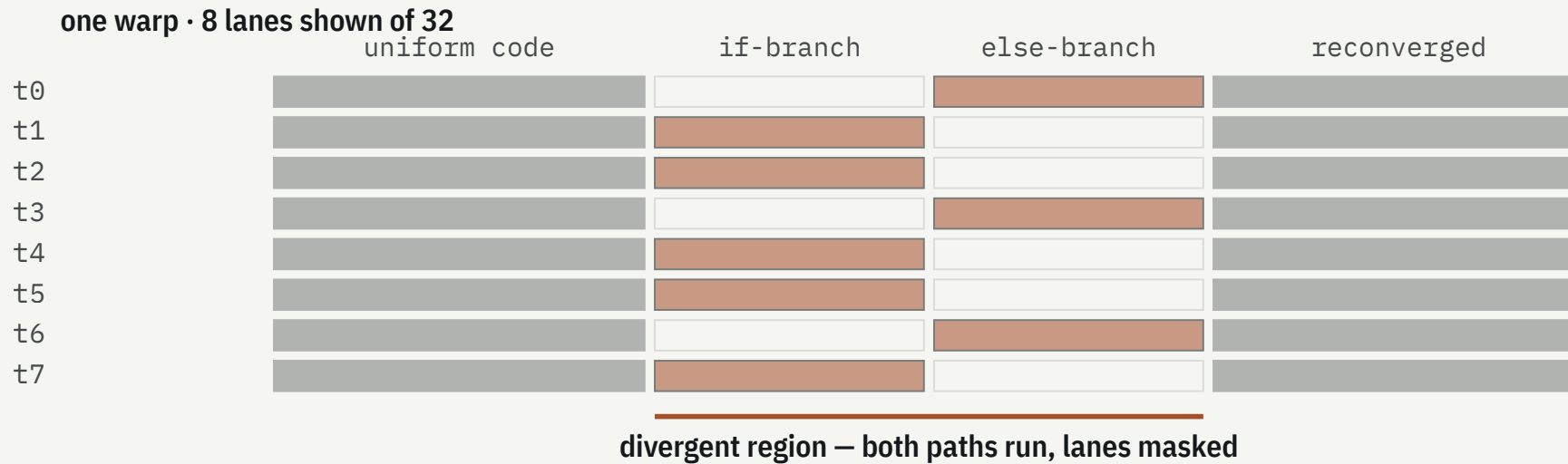
THREE MECHANISMS, THREE CONSEQUENCES

What a GPU actually is

- Threads are scheduled in lock-step groups — **warps**, typically 32. All lanes execute the same instruction.
- **Divergence.** If lanes take different branches, both paths execute with inactive lanes masked.
- **Coalescing.** Consecutive threads should touch consecutive addresses. Layout choices are coalescing choices.
- **Occupancy.** Latency is hidden by resident warps; residency is limited by registers and shared memory. Tile sizing is a three-way negotiation.

THE COST OF A BRANCH

Both sides run. Inactive lanes still occupy their slot in time.



Faint cells are lanes that are switched off but still occupy their slot in time.

This is why unstructured sparsity, early exits and any data-dependent branch are expensive on a GPU – and why N:M sparsity, whose pattern is fixed at compile time, is not.

Why unstructured sparsity, early exits and any data-dependent branch are expensive here – and why **N:M** sparsity, fixed at compile time, is not.

BREAK

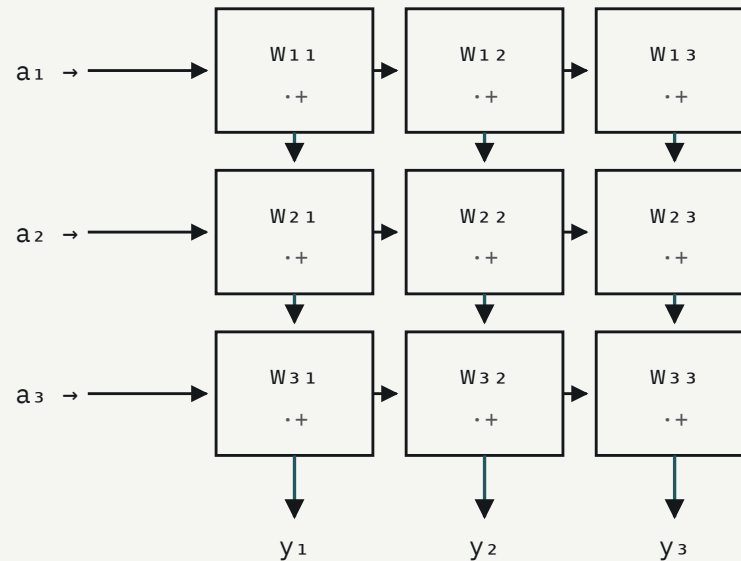
Five minutes.

Next: dataflow accelerators, beyond-digital hardware, and deployment.

TRACE IT CYCLE BY CYCLE

N^2 multiply–accumulates per cycle, $2N$ operands fetched

weight-stationary systolic array – each cell holds one weight for the whole tile



partial sums accumulate downward; results emerge after pipeline fill

per cycle, every cell:

1. take activation from the west
2. take partial sum from the north
3. $\text{psum} += w \cdot a$
4. pass activation east, psum south

the weight never moves

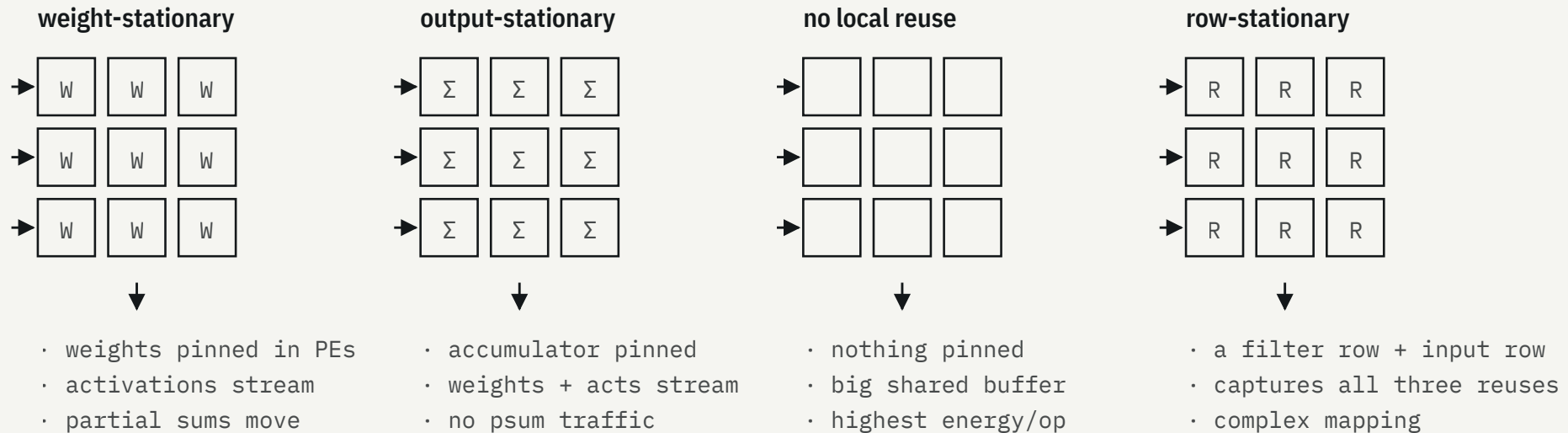
$N \times N$ cells do N^2 MACs per cycle
while reading only $2N$ operands
from the array edge

→ arithmetic intensity scales with N

external memory traffic falls by N ,
which is the whole point of the design

THE TAXONOMY THAT LETS YOU READ A DATASHEET

Which reuse does it capture?



Energy is dominated by data movement, so the right question about an accelerator is not how many multipliers it has but which reuse its dataflow captures – and whether your layer has that kind of reuse.

A depthwise layer has almost no weight reuse to capture. Weight-stationary hardware will not save it.

A depthwise layer has almost **no weight reuse** to capture. Weight-stationary hardware will not save it.

AT THE MICROCONTROLLER END

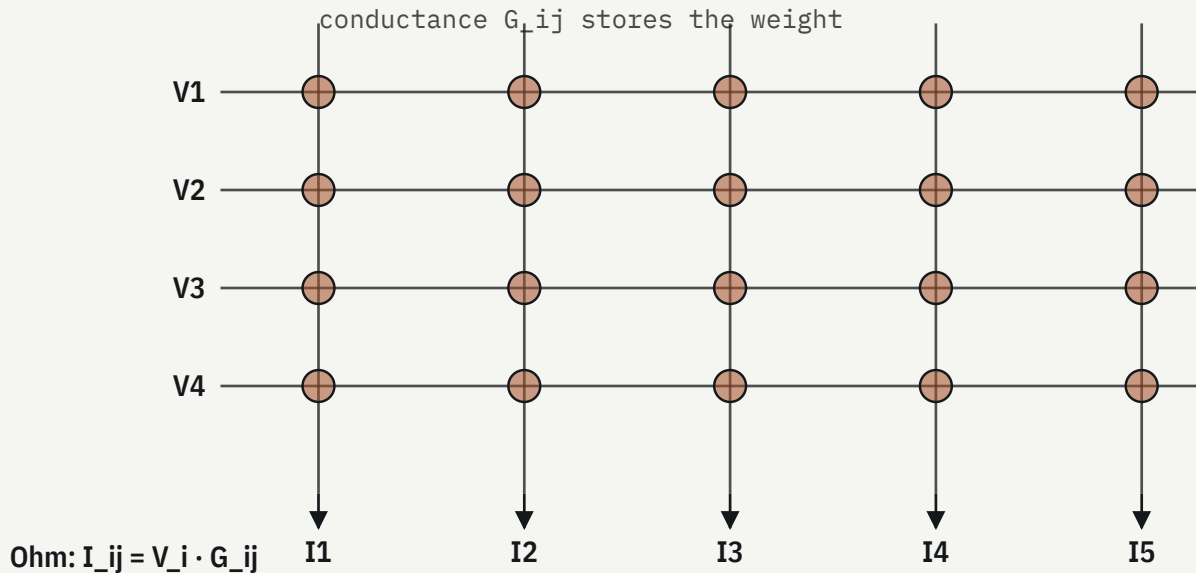
MicroNPUs, and the operator-coverage trap

- A small accelerator beside a Cortex-M core, executing a **restricted operator set** in int8 from a command stream prepared offline by a vendor compiler.
- Anything unsupported **falls back to the CPU** — often with a layout conversion in each direction.
- One unsupported operator in the middle of a graph can cost more than every optimisation elsewhere.
- **The vendor compiler, not your training framework, decides whether your model is fast.**

Check the operator-support list *before* designing the architecture, not after.

TWO PHYSICAL LAWS INSTEAD OF A DATAPATH

Analog in-memory computing



Kirchhoff: $I_j = \sum_i V_i \cdot G_{ij}$

one matrix-vector product, one step, weights never move

But: the ADCs at the bottom dominate the energy, conductances drift with time and temperature, writes are slow and wear the device, and the effective precision is a few bits.

A 64-core PCM chip in 14 nm reported up to **63.1 TOPS** and **9.76 TOPS/W** in low-precision mode, with near-software-equivalent accuracy on ResNet and LSTM workloads (*Le Gallo et al., Nature Electronics, 2023*).

HONESTLY

Neuromorphic and spiking systems

- Information in the **timing of sparse events**; computation only when events arrive.
- A natural fit for always-on sensing with a low event rate — event cameras, spiking audio front ends.
- Real demonstrated advantages in latency and energy for sparse, temporally structured signals.
- **Open problems:** training (surrogate gradients are improving, not routine), no standard software stack, and a shortage of tasks where the sparsity assumption clearly holds.

CHOOSING

No platform wins every axis

five dots = best on that axis · a teaching judgement, not a measurement

	energy efficiency	model flexibility	toolchain maturity	low unit cost	latency determinism	peak throughput
MCU (Cortex-M)	● ● ● ○ ○	● ● ● ● ●	● ● ● ● ●	● ● ● ● ●	● ● ● ● ●	● ○ ○ ○ ○
MCU + NPU	● ● ● ● ○	● ● ● ○ ○	● ● ● ● ○	● ● ● ● ○	● ● ● ● ○	● ● ○ ○ ○
DSP	● ● ● ● ○	● ● ● ○ ○	● ● ● ○ ○	● ● ● ● ○	● ● ● ● ●	● ● ○ ○ ○
mobile SoC + NPU	● ● ● ● ○	● ● ● ○ ○	● ● ● ○ ○	● ● ○ ○ ○	● ● ○ ○ ○	● ● ● ● ○
embedded GPU	● ● ○ ○ ○	● ● ● ● ●	● ● ● ● ●	● ○ ○ ○ ○	● ● ○ ○ ○	● ● ● ● ●
FPGA	● ● ● ● ○	● ● ● ● ○	● ● ○ ○ ○	● ● ○ ○ ○	● ● ● ● ●	● ● ● ● ○
edge ASIC / TPU-class	● ● ● ● ●	● ● ○ ○ ○	● ● ● ○ ○	● ● ● ○ ○	● ● ● ● ○	● ● ● ● ○
analog / neuromorphic	● ● ● ● ●	● ○ ○ ○ ○	● ○ ○ ○ ○	● ● ○ ○ ○	● ● ● ○ ○	● ● ● ○ ○

Read down a column to see why there is no single "best" platform: each axis is won by a different row.

PUTTING IT TOGETHER

Choosing a platform

IF YOUR BINDING CONSTRAINT IS...	CONSIDER	WATCH OUT FOR
Microwatts, years on a cell	MCU, duty-cycled, cascade front end	Wake-up energy dominating compute
Milliwatts, int8 CNN, tight unit cost	MCU + microNPU	Unsupported operators falling back to CPU
Single-stream latency	Mobile SoC / embedded module	Batch-1 occupancy collapse on the GPU
Many concurrent streams	Embedded GPU module	Thermal throttling changing your benchmark
Deterministic microsecond latency	FPGA	Development cost and model rigidity
Volume > 10 ⁶ , fixed model	ASIC	Non-recurring engineering and the schedule

In practice **ecosystem maturity** – compiler quality, operator coverage, debugging tools – decides more projects than peak performance, and it is the factor most often omitted from comparisons.

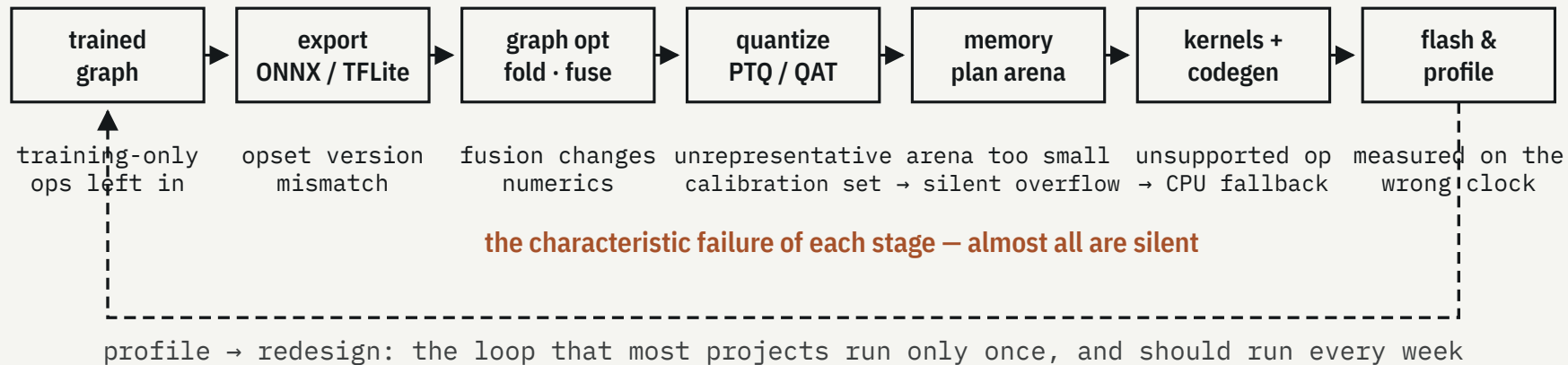
02

From graph to binary — and after

Every stage can break the model silently.

SEVEN STAGES, SEVEN SILENT FAILURES

The deployment pipeline



Parity test at every arrow: run the same input through both sides and compare per-tensor.

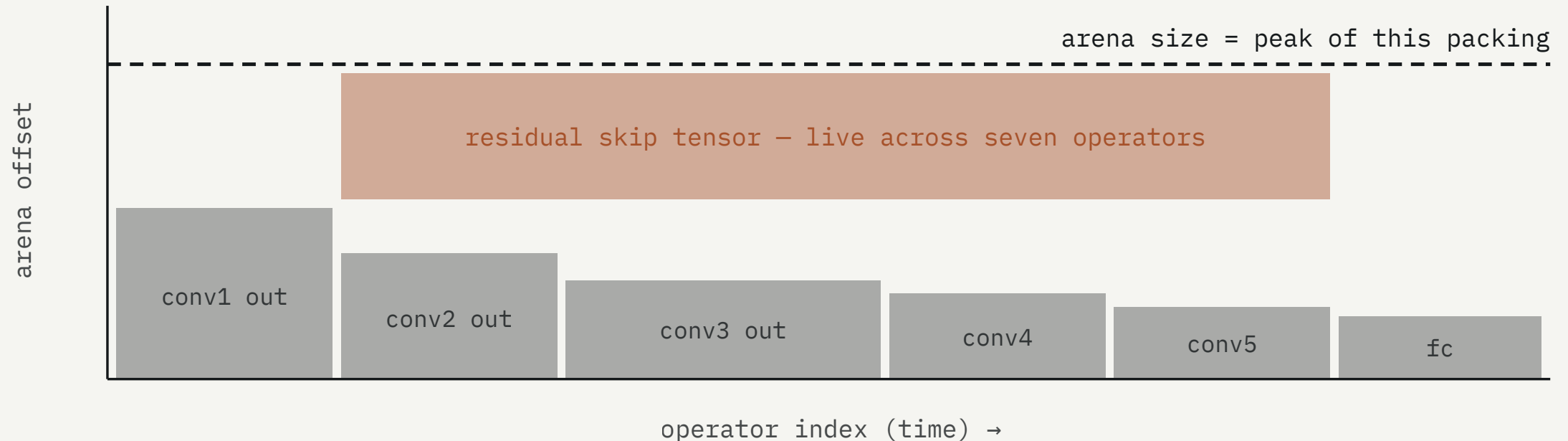
float reference → exported graph → quantized graph → on-device run. The first arrow where SQNR collapses is the bug. Without this, a 4-point accuracy drop is a week of guessing; with it, it is twenty minutes.

Parity-test at every arrow: same input, both sides, compare per tensor. **The first arrow where SQNR collapses is the bug.**

OFFLINE PACKING

Memory arena planning

tensors as boxes: height = bytes, width = liveness interval



A long-lived skip tensor raises the arena for the entire network – which is why residual topology is a memory decision.

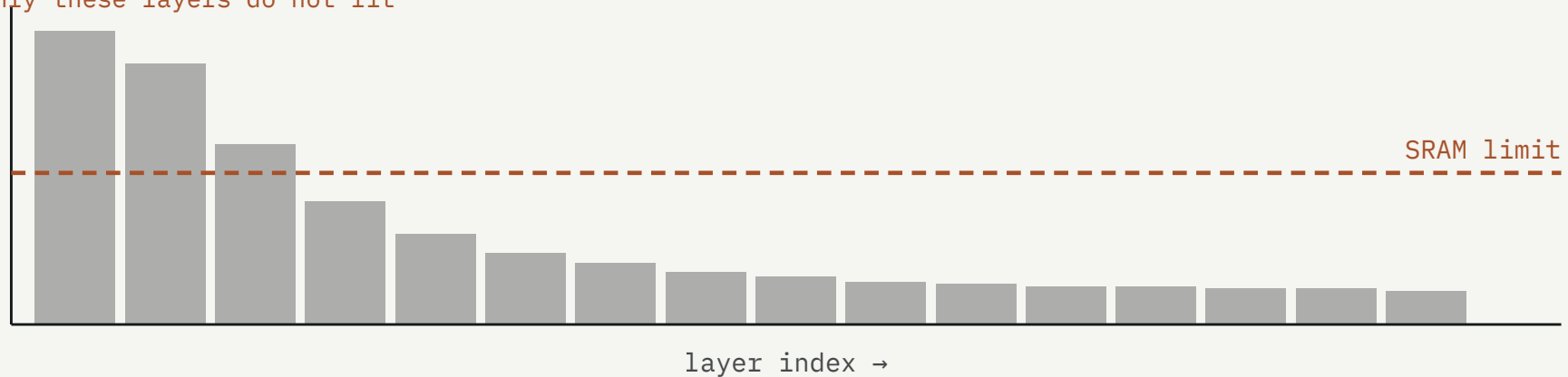
A **long residual connection** raises the arena for the whole network – which is why residual topology is a memory decision.

THE OBSERVATION THE WHOLE PAPER RESTS ON

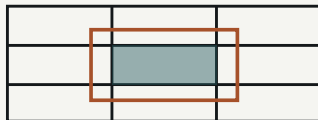
Peak memory is a spike, not a plateau

per-layer peak activation memory – the shape that constrains everything

only these layers do not fit



Patch-based inference: run the early layers on spatial tiles, keeping only one patch plus its halo live.



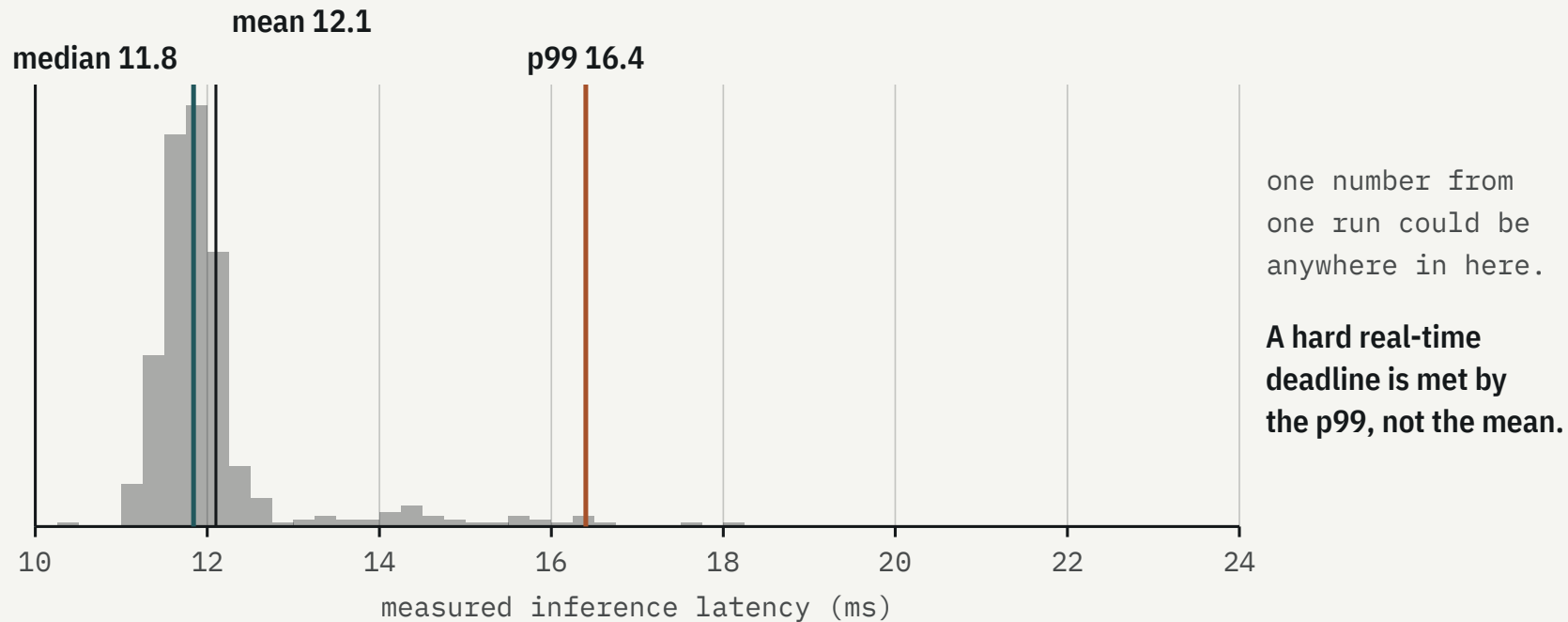
one patch
+ halo the convolutions need

peak memory ↓ 4–8× · overlapping halos are recomputed
redistributing the receptive field shrinks the halo, recovering most of the cost

Only a handful of early layers do not fit – and the whole network is sized for them.

METHODOLOGY FOR EVERYONE

A measurement is a distribution, not a number



one number from
one run could be
anywhere in here.

**A hard real-time
deadline is met by
the p99, not the mean.**

430 repetitions on one device. The second mode is cache state; the tail is interrupt service.

A paper reporting "11.8 ms" has reported the **left edge** of this picture.

HOW TO MEASURE ON AN MCU

A defensible latency measurement

1. Read the cycle counter (DWT -> CYCCNT) around the kernel and subtract the measurement overhead.
2. Report **cycles**, not milliseconds — the number then survives a clock change.
3. Decide and state whether caches are warm. Either is fine; silence is not.
4. Repeat, and report the **median and the interquartile range**.
5. Say whether interrupts were enabled.

A single run on a system with interrupts enabled is not a measurement.

TESTING

Four practices that pay for themselves in a week

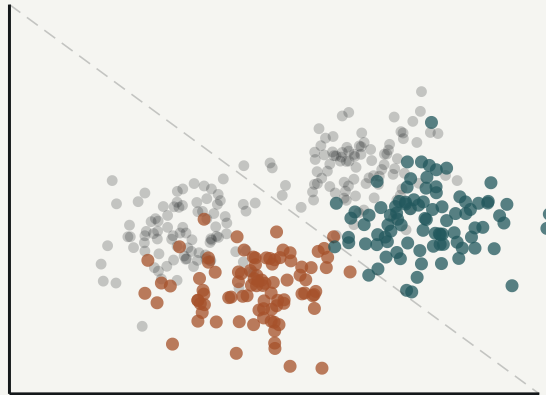
1. **Per-layer parity.** Identical inputs through the float reference and the deployed graph; dump every intermediate; report per-layer SQNR.
2. **Golden vectors.** Freeze inputs and expected outputs into the firmware test suite, so a toolchain upgrade cannot silently change behaviour.
3. **Hardware in the loop.** CI that flashes real boards. Emulators do not reproduce timing, DMA, interrupts or wait states.
4. **Resource assertions.** Fail the build if flash, arena size or worst-case latency exceeds the budget from Session 1.

Budgets that are not enforced are not budgets.

NAME IT BEFORE YOU FIX IT

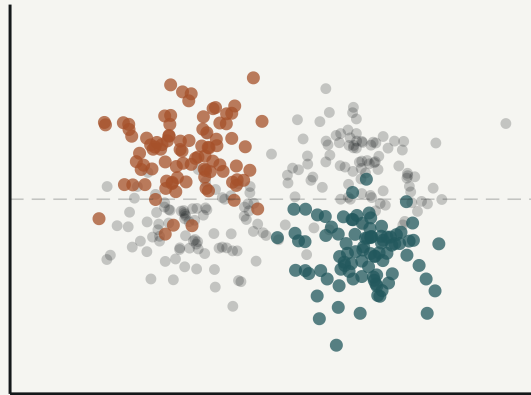
Three drifts, three different remedies

covariate shift



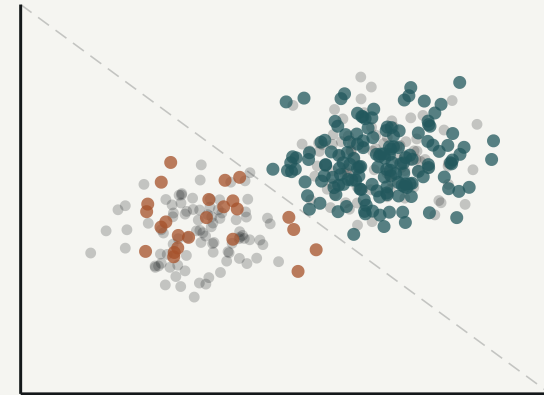
inputs moved, boundary did not

concept drift



the boundary itself moved

label shift



only the class mix changed

faint = deployment day one · solid = eight months later · dashed = the model's boundary

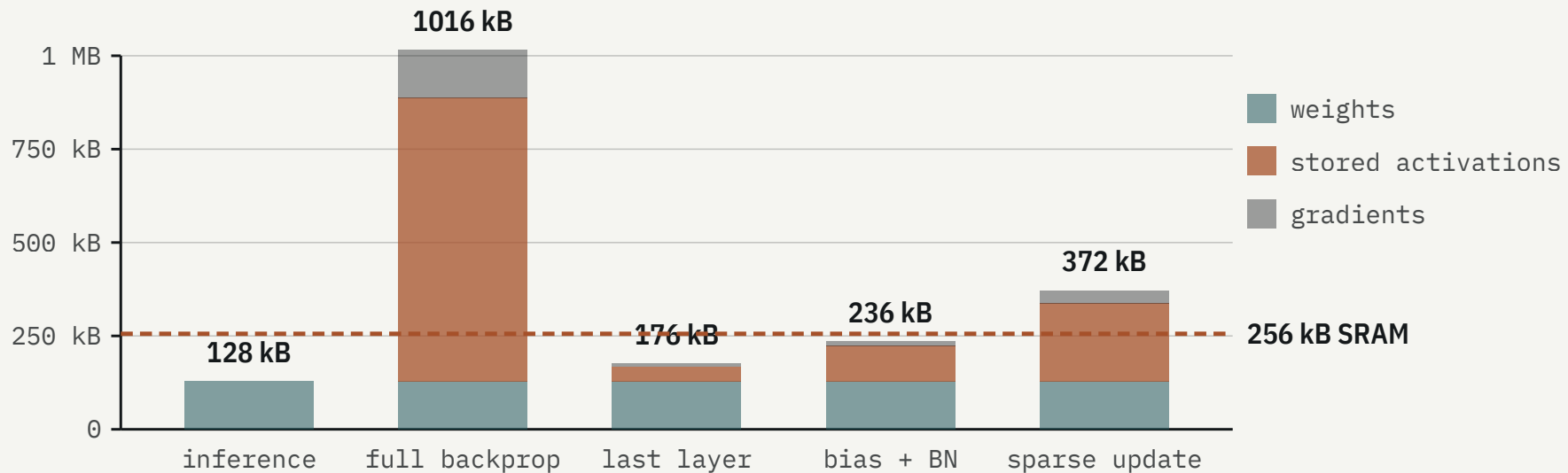
Fix by recalibration**Needs new labels****Correctable analytically**

Distinguishing them is what decides whether you need a technician visit or a prior adjustment.

Covariate → recalibrate. Concept → **needs new labels**. Label shift → correctable in closed form *if the model is calibrated*.

THE QUANTITATIVE REASON

Training memory is activations, not weights



Backpropagation must keep every forward activation until its gradient is used – so training memory is set by the activations, not by the parameter count.

Every practical on-device-training method is a way of storing fewer activations.

Backpropagation must retain every forward activation until its gradient is used. **Every practical method is a way of storing fewer of them.**

THREAT MODEL

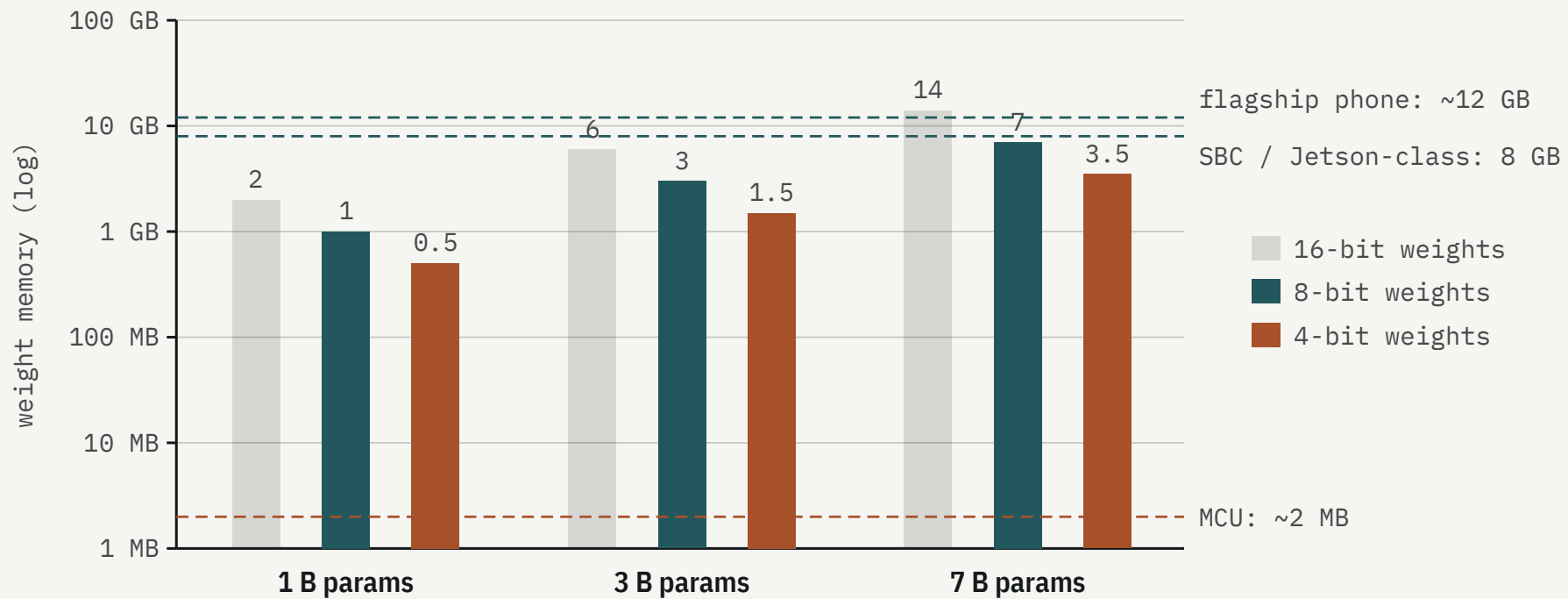
Edge deployment improves data privacy and worsens model confidentiality

THREAT	WHAT THE ATTACKER GETS	MITIGATION
Model extraction	the weights, by reading flash or by querying	Secure boot, encrypted flash, read-out protection, rate limiting
Membership inference	whether a record was in the training set	Differential privacy; avoid over-fitting
Adversarial inputs	controlled misclassification	Adversarial training, input plausibility checks
Physical / sensor spoofing	injected signals treated as real — ultrasonic audio, projected light	Multi-sensor consistency, band-limiting in the analog chain
Power / EM side channel	architecture and sometimes weights, from the current trace	Constant-time kernels, masking, noise injection — all costly
Fault injection	skipped instructions, corrupted results	Redundant computation, output plausibility checks

The side-channel row is the one specific to this domain: physical access means the current trace is correlated with the computation.

FRONTIER

Language models at the edge are a quantization story first



Weights only (params × bits / 8). KV cache, activations, runtime and OS come on top.

LANGUAGE MODELS AT THE EDGE

Be precise about what exists

Real

- 1–3 B parameter models on phones and embedded modules
- 4-bit weight quantization — AWQ, GPTQ
- KV-cache management, speculative decoding
- Small task-specific models distilled from large ones

Not real

- Language models on kilobyte-class microcontrollers
- The binding constraint is **weight memory and bandwidth per generated token**
- No amount of clever kernel work removes it
- The productive embedded pattern is still Session 1's **cascade**

SESSION 5

Presentations — what to prepare

One landmark paper per presenter, one topic area each (A quantization · B pruning and distillation · C efficient architectures and NAS · D TinyML systems and hardware). 15 minutes + 5 minutes discussion.

Why: Claim in one sentence with its scope; the mechanism re-derivable; the load-bearing experiment; one real limitation; what it means on a constrained device.

Slides to the lecturer the evening before. Discussants: one prepared question.

IF YOU REMEMBER ONE THING

Ask where the bytes come from before asking how fast the arithmetic is. The roofline says which layers an accelerator can help, the toolchain says which it can run at all, and only a measurement on the device, with its method stated, says what you actually have.

BEFORE YOU GO

Think about these

- Q1 An NPU vendor reports 4 TOPS/W. When could the same network be more energy-efficient on the MCU core alone?
- Q2 Why do depthwise convolutions run at a small fraction of an accelerator's peak? What would a hardware designer change?
- Q3 Which post-deployment problem — drift, updates, security — is least addressed by research, in your view?